

Reviewer Pack — Plethysm Coefficient Sum Gap in Symmetric Powers of Permanen...

Entry f1537130764a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 06:08:20 UTC

Plethysm Coefficient Sum Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: f1537130764a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 06:08:20 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Plethysm Coefficients) **Field B** (complexity object): Monotone Circuit Size for Permanent Polynomial

Statement:

For a homogeneous polynomial f of degree d , let $P(f)$ denote the sum of plethysm coefficients in its decomposition into symmetric powers of the determinant polynomial. Then $P(\text{perm}_n) > P(\text{det}_m)$ for all $m < n^{\{1.5\}}$, and $P(f)$ is preserved under linear substitutions.

Rationale (proposer's reasoning):

Plethysm coefficients capture combinatorial symmetries in polynomial decompositions. By linking them to monotone circuit size, we might expose algebraic structure that distinguishes permanent's inherent complexity from determinant's tractability.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a43f1bbedef4533a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, permutations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def isqrt(n):
        x = n
        y = (x + 1) // 2
        while y < x:
            x, y = y, (y + x // y) // 2
        return x

    def hook_length_formula(shape):
        m, n = len(shape), len(shape[0])
        total = 1
        for i in range(m):
            for j in range(n):
                h = shape[i][j]
                total *= (m + i - h) * (n + j - h)
                total //= (i + 1) * (j + 1)
        return total

    def plethysm_coefficient_sum(poly, n):
        m = len(poly)
        total = 0
        for shape in combinations(range(m), n):
            coeff = hook_length_formula(shape)
            total += coeff * poly[shape]
        return total

    def perm_poly(n):
        return [1] + [math.factorial(i) for i in range(1, n)]

    def det_poly(n):
        return [1] + [(-1)**i * math.comb(n, i) for i in range(1, n)]

    n = random.randint(5, 40)
```

```

m = isqrt(n ** 1.5) - 1

perm_plethysm = plethysm_coefficient_sum(perm_poly(n), n)
det_plethysm = plethysm_coefficient_sum(det_poly(m), m)

metric_name = "Plethysm Coefficient Sum Gap"
metric_value = perm_plethysm - det_plethysm
instances_tested = 1
conjecture_holds = perm_plethysm > det_plethysm
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 97, 3)) # Default to first 30 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4015c97.py", line 76, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4015c97.py", line 54, in run_trial
    perm_plethysm = plethysm_coefficient_sum(perm_poly(n), n)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4015c97.py", line 41, in plethysm_coef

```

```
coeff = hook_length_formula(shape)
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e4015c97.py", line 28, in hook_length_f
m, n = len(shape), len(shape[0])
^^^^^^^^^^^^^^^^
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in hook_length_formula, preventing evaluation of conjecture | next: Fix type handling in hook_length_formula and revalidate plethysm coefficient calculations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	20121
2	preregistration	ollama_remote	qwen3:8b	0	0	24123
3	novelty	ollama_remote	qwen3:8b	0	0	21549
4	novelty	ollama_remote	qwen3:8b	0	0	18362
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10363
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8920
7	judge	ollama_remote	qwen3:8b	0	0	22330

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 125769 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in *research/audit/f1537130764a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/f1537130764a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/f1537130764a.tar.gz* (if generated)